

Guia de Implementação - Validação com Zod em Endpoints

Padrão de Implementação

Todos os endpoints POST/PUT/PATCH devem usar Zod para validar input. Aqui está o padrão:

1. Criar Schema (lib/validation-schemas.ts)

```
export const createYourResourceSchema = z.object({
  name: z.string().min(2).max(100),
  email: z.string().email(),
  // ... mais campos
});

export const updateYourResourceSchema = createYourResourceSchema.partial();
```

2. Usar em Endpoint (app/api/...)

```
import { validateRequest, createAPIHandler } from '@lib/api-utils';
import { createYourResourceSchema } from '@lib/validation-schemas';

export const POST = createAPIHandler(async (request: NextRequest) => {
  // Validação automática
  const data = await validateRequest(request, createYourResourceSchema);

  // Autenticação
  const session = await requireAuth(request);

  // Lógica do endpoint
  const result = await prisma.yourModel.create({ data });

  return NextResponse.json(result);
});
```

3. Benefícios

-  Validação automática
-  Mensagens de erro consistentes
-  Type safety
-  Proteção contra injection attacks
-  Código mais limpo

Endpoints Atualizados

Categoria A - IMPLEMENTADO (Exemplo)

- [] POST /api/gestor/categories
- [] PUT /api/gestor/categories/[id]
- [] POST /api/gestor/partners

- ☐ PUT /api/gestor/partners/[id]
- ☐ POST /api/gestor/authors
- ☐ PUT /api/gestor/authors/[id]

Categoria B - FALTANDO

Todos os outros 53 endpoints POST/PUT/PATCH

Progresso

- ☐ Categoria A (6/59)
- ☐ Categoria B (53/59)
- ☐ Total: 0% → 100%

Como Expandir

1. Adicione schema em `lib/validation-schemas.ts`
2. Use `validateRequest()` no início de cada handler POST/PUT/PATCH
3. Remova manualmente validações antigas (`if (!name)`)
4. Teste no Postman/curl

Checklist

- ☐ Todos POST/PUT/PATCH usam `validateRequest()`
- ☐ Mensagens de erro são consistentes
- ☐ Testes passam
- ☐ TypeScript compila sem erros